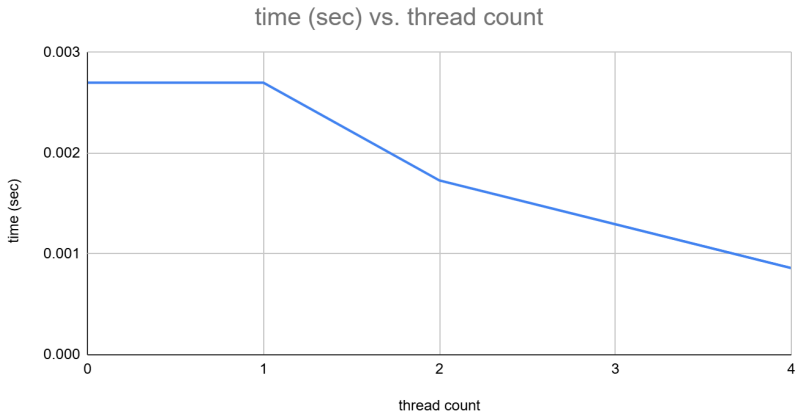
Homework 7 - Vector sum

Note: if nothing specified then intel/24 and no optimisation specified. All exercise were done with idev session with 3 Nodes (scaling study upto 128 threads)

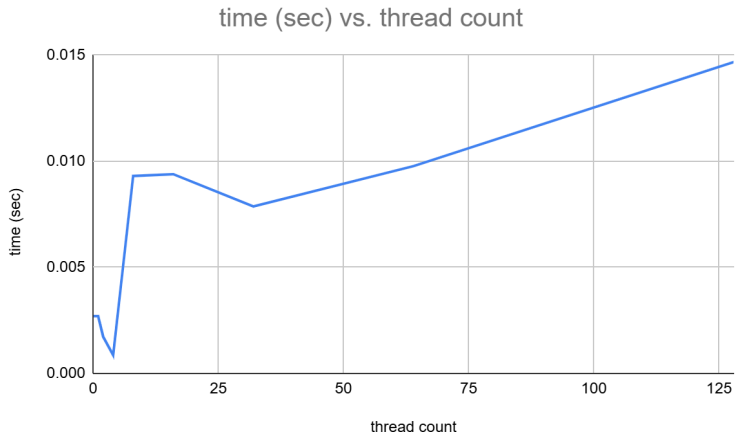
1. Sequentially for vectorsize 1000 : t = 1.9e-4 seconds
2. 1 thread for vectorsize 1000: t = 1.9e-4 seconds
3. 2 thread timings :

Vector size	2 thread (sec)	1 thread(sec)	Comment
1000	3.1e-4	1.9e-4	2 thread slower
10000	1.7e-3	2.7e-3	2 thread faster

Strong scaling study for vectorsize 10000 and no optimisation -



But if we check till 1024 cores then -



How to improve performance -

1. Swap the two for loops depending on vector size and nloop. (if nloop >> vectorsize swap but other wise not trivial)
2. Use optimisation when compiling (this will handle the confusion of swapping etc. Multiversioning,vectorization etc)
3. Use of optimal number of threads depending on vectorsize (too many threads increases cache misses and other overheads making overall process slower)
4. Choice of compiler (gcc vs icc). Usually depends on the processor. Since intel processor icc better mostly
5. Load balancing as coefficients are increasing and uneven time if naive splitting of compuatations

Other observations and understandings after playing around with various vectorsizes, gcc vs icc (23 vs 24) and icc 24 (no optimisation vs -O1,-O2,O3) -

Note: if nothing specified then intel/24 and no optimisation specified. All exercise were done with idev session with 3 Nodes (scaling study upto 128 threads) and **vectorsize 1000**

- ICC 24 faster than GCC 13 in sequential but for parallel and high number of threads gcc showed better performance
- Icc -O3 > -O2 > no optimisation > -O1 in terms of sequential performance
- Icc -O3 > no optimisation > -O2 > -O1 in term of parallel performance (high number of threads)
- ICC 23 and 24 very similar . 24 performed better for higher threads
- When i used optimisation i observed all threads giving output. So i used omp barrier and then omp master